Guardefy

Origami OPAL Vault

Audit Report

prepared by Pavel Anokhin (@panprog)

December 2, 2025

Version: 2

Guardefy

Contents

1. About Guardefy and @panprog

Pavel Anokhin or **@panprog**, doing business as Guardefy, is an independent smart contract security researcher with a track record of finding numerous issues in audit contests, bugs bounties and private solo audits. His public findings and results are available at the following link:

https://audits.sherlock.xyz/watson/panprog

2. Disclaimer

Smart contract audit is a time, resource and expertise bound effort which doesn't guarantee 100% security. While every effort is put into finding as many security issues as possible, there is no guarantee that all vulnerabilities are detected nor that the code is secure from all possible attacks. Additional security audits, bugs bounty programs and on-chain monitoring are strongly advised.

This security audit report is based on the specific commit and version of the code provided. Any modifications in the code after the specified commit may introduce new issues not present in the report.

3. Scope of the audit

The code at the following link was reviewed:

https://github.com/TempleDAO/origami

commit hash: 72481ffa5be3d3693216bfe6f2cc70303c4f86c2

Files in scope:

```
./contracts/investments/opal/OpalVault.sol
./contracts/investments/opal/OpalManager.sol
./contracts/investments/opal/OpalAdapterFactory.sol
./contracts/investments/opal/adapter/OpalAdapterBase.sol
./contracts/investments/opal/adapter/OpalAdapterSpotAssets.sol
./contracts/investments/opal/adapter/OpalAdapterAaveV3.sol
./contracts/investments/opal/adapter/OpalAdapterMorpho.sol
./contracts/investments/opal/adapter/OpalAdapterEuler.sol
```

4. Audit Timeline

Audit start: November 17, 2025

Audit report delivered: December 1, 2025

Fixes reviewed: December 2, 2025

5. Findings

5.1. High severity findings

None found.

5.2. Medium severity findings

None found.

5.3. Low severity findings

5.3.1. [L-1] `OpalVault.maxExitWithToken` and `maxExitWithShares` always revert if all adapters return `type(uint256).max` for all tokens.

When calculating max exit shares amount, adapters (and manager) can return a value of `type(uint256).max`, which means there is no global exit limit. However, if manager returns it (meaning all underlying adapters return it for all tokens), transaction will always revert in `_leastOfMaxExitWithShares` when trying to calculate pre-fee shares amount:

```
// Calculate the shares before the exit fee is taken on the shares.
// Inverse of the fees being taken in _previewExitWithShares();
maxShares = maxShares.inverseSubtractBps(exitFeeBps(), OrigamiMath.Rounding.ROUND_UP);
```

When `maxShares` is uint256 max, fees shouldn't be applied to it since it's a magic number.

The same issue also happens when calculating max join amount, but it has no impact since the amount is reduced (rather than increased) and doesn't cause overflow (and later in the flow it's reversed to come back to max value again):

```
// Calculate the shares before the exit fee is taken on the shares.
// Inverse of the fees being taken in _previewJoinWithShares();
maxShares = maxShares.subtractBps(joinFeeBps(), OrigamiMath.Rounding.ROUND_DOWN);
```

Proof of concept

Slight modification of maxExit test, setting fees to non-0:

```
    function test_maxExitWithToken_noSupplyCap_noAdapterCap() public {
        uint256 adapterCap = type(uint256).max;
        vm.prank(origamiMultisig);
        manager.setFees(100, 100);

        for (uint256 i; i < adapters.length; ++i) {
            vm.mockCall(
                address(adapters[i]),
                abi.encodeWithSelector(IOpalAdapter.maxExit.selector),
                abi.encode(mkArray(adapterCap), mkArray(adapterCap))
            );
        }
        assertEq(vault.maxExitWithToken(ASSET1, address(1)), 0);
    }
```

This test reverts when trying to calculate max exit.

Likelihood

Low. Most adapters will have some set exit limit for liability tokens, which is enough to avoid the revert. The only current adapter with max exit returning `type(uint256).max` is `SpotAssets`. If this is the only adapter, the opal vault will always revert on max exit calls.

Impact

Denial of service for all users trying to calculate max exit amount.

Possible mitigation

Apply fees to `maxShares` only if it's not `type(uint256).max`.

Status: Fixed.

Fix Review: Fixed as suggested.

5.3.2. [L-2] Joining/exiting with the value returned by `maxJoin`/`maxExit` can revert in some cases due to distribution algorithm forcing higher amount to the last adapter(s) from rounding error.

When calculating max join / max exit amounts, manager assumes pro-rata distribution of the token amounts to underlying adapters, rounding down any corresponding amounts.

However, the actual distribution algorithm is slightly different. It always distributes full token amount between adapters. In case of rounding errors, it has to round it down for some adapters and round up for the other adapters. Moreover the algorithm is such that it favors rounding down for the first adapters, which can lead to the last adapter(s) exceeding the initial limit.

Example:

```
3 adapters balances: 4 + 2 + 3 = 9
Adapter 3 maxJoin = 2
Manager calculates maxJoin for the whole vault as: 2 * 9 / 3 = 6
If we calculate fractional pro-rata allocation, it's:
Adapter 1: 6 * 4 / 9  = 2.67
Adapter 2: 6 * 2 / 9 = 1.33
Adapter 3: 6 * 3 / 9 = 2
Total: 2.67+1.33+2 = 6
```

As can be seen from the example, distribution of allocation to adapters is fractional. If we want to distribute exact integer allocated amount (6), since each adapter's allocation is also integer, it's inevitable that some adapters will exceed their fractional allocations.

But the distribution algorithm used goes even further and tends to round down first adapters, meaning that in order to distribute the exact integer allocation, the last adapter(s) are forced their allocation to exceed their pro-rata share, sometimes by more than 1. In the example above, the distribution algorithm distributes it as:

```
Adapter 1 = 6 * 4 / 9 = 2.67 = 2
Adapter 2 = (6 - 2) * 2 / (9 - 4) = 4 * 2 / 5 = 1.6 = 1
Adapter 3 = 3
```

For Adapter 3, max join amount is 2, but the allocation is 3, exceeding the limit and possibly reverting the transaction, when trying to actually join with 3.

Some simulation testing reveals the following details:

- The issue happens only with 3+ adapters for the same token
- For small balance and max join / exit amounts (< 100), the probability for the last adapter to exceed its max limit is 50%+
- When there are more than 3 adapters, any adapter starting from the 3rd can exceed the max join / exit limit, not just the last one. For example, with 4 adapters, 3rd adapter has 1.5% chance to exceed the limit, 4th adapter has 58% chance to exceed its limit. With 5 adapters, 3rd adapter has 0.1% chance, 4th adapter has 6% chance, 5th adapter has 64% chance.
- Actual allocation can exceed max adapter limit by at most (number of adapters – 2).
- For large balances and max join / exit amount (> 1e6), the probability to exceed the limit drops to below 1%, so this is more an issue of small numbers. The probability is still high if balances are close to each other even for large numbers.

Proof of concept

https://gist.github.com/panprog/b7712a08e7915cf6e5845f513f70e1a2

This test showcases revert with max join amount using the example above.

Likelihood

Low to medium depending on situation.

Impact

Denial of service when trying to join/exit with max join / max exit amounts returned by the opal vault.

Possible mitigation

Since the exceeding of maximum join/exit amount is coming from rounding errors, and max rounding error is at most 1 per adapter, total rounding error is always less than the number of adapters. For the last adapter total rounding error excludes the last adapter itself (as it's simply remainder of allocation), thus the max theoretical exceed over max amount is (number of adapters – 1). If the initial max amount returned from adapter is reduced by (number of adapters – 1) before proceeding with calculations, this is enough to ensure that actual allocation never exceeds the max limit. In simulation tests actual allocation never exceeds limit by more than (number of adapters – 2).

Status: Fixed.

Fix review: Fixed as described above.

5.3.3. [L-3] Shared `maxJoin` / `maxExit` amounts can only be calculated if all adapter assets are shared, causing reverts if trying to join / exit with max amounts in some situations.

When adapters join / exit in the same protocol, their global (protocol-level) `maxJoin` / `maxExit` limits are shared, which is taken into account by using the adapter's `groupId()` function: if it's the same for 2 adapters, **all** tokens in these adapters are considered shared.

The issue is that some protocols (such as Euler) allow different collateral and/or liability tokens to be used in different adapters. This means that some (but not all) tokens in the

protocol can be shared. Currently it is not possible to control per-token sharing, only per-adapter.

Example:

- Euler has 2 USDC vaults: VaultUSDC1 and VaultUSDC2 (perhaps different LTV requirements, risk settings etc)
- Euler has has SUSDE vault.
- Adapter1: asset = VaultUSDC1, liability = SUSDE
- Adapter2: asset = VaultUSDC2, liability = SUSDE

Currently the Euler's groupId is hash of both tokens, so if either token is different, adapter's group is different. This means both tokens will be calculated as separate, but since SUSDE is shared, it won't share max join / exit and thus will return incorrect max join / exit value.

With the current per-adapter groupId setting it's impossible to set it up correctly to handle one token (asset) as separate and the other (liability) as shared.

Likelihood

Medium.

Impact

Denial of service when trying to join/exit with max join / max exit amounts returned by the opal vault.

Possible mitigation

Use per-token groupId instead of per-adapter.

Status: Fixed.

Fix review: Fixed as described above.

5.4. Informational severity findings

5.4.1. [I-1] Incorrect `maxJoin` / `maxExit` amounts returned for Aave pools in isolation mode.

Some Aave V3 vaults have "isolation mode" turned on, which has a separate borrowing cap not accounted for in the Aave adapter's `maxJoin` / `maxExit` calculations. These functions will likely return incorrect amount if such vaults are used (higher than real amount) and will revert when trying to actually join with this amount.

Impact

Denial of service for users who try to join/exit with amounts returned by opal vault `maxJoin` / `maxExit` functions with Aave V3 isolation mode pools.

Possible mitigation

Calculate max join / exit amounts correctly or make sure not to use such Aave V3 pools.

Status: Fixed

Fix Review: Isolation mode is now accounted for and max join/exit amounts calculated correctly.